

# Django REST Framework

Day 01 Notes — Models, Serializers, Views, ViewSets, Routers & URLs

---

These notes cover the five core building blocks of a Django REST Framework (DRF) API: **Models**, **Serializers**, **Views / ViewSets**, **Routers**, and **URLs** — their definitions, basic syntax, sample code, why each one is needed, and how they connect together in a single request/response workflow.

## 1 Model

**Definition:** A Model is a Python class (defined in `models.py`) that represents a single database table. Each attribute of the class becomes a column/field in that table. Django's ORM (Object Relational Mapper) uses the model to generate SQL and lets you interact with the database using Python objects instead of raw SQL.

### Basic Syntax

```
from django.db import models

class ModelName(models.Model):
    field_name = models.CharField(max_length=100)
    field_name2 = models.IntegerField()

    def __str__(self):
        return self.field_name
```

*General shape of a model class*

### Sample Code

```
# models.py
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.IntegerField()
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

*Student model with common field types*

#### Why we use Models

Models are the single source of truth for your data structure. They let Django auto-generate the database schema through migrations, give you a Python-friendly query API (`Student.objects.all()`) instead of raw SQL, and are the object that a Serializer later converts into JSON.

## 2 Serializer

**Definition:** A Serializer converts complex data such as Django model instances or querysets into native Python datatypes (dict/list) that can then be rendered into JSON/XML — this is called **serialization**. It also does the reverse: it parses incoming JSON data, validates it, and converts it back into a model instance — this is called **deserialization**. It is defined in `serializers.py`.

### Basic Syntax

```
from rest_framework import serializers

class NameSerializer(serializers.ModelSerializer):
    class Meta:
        model = ModelName
        fields = '__all__' # or a list like ['id', 'field_name']
```

*ModelSerializer — most common type, built on top of a Model*

```
from rest_framework import serializers

class NameSerializer(serializers.Serializer):
    field_name = serializers.CharField(max_length=100)

    def create(self, validated_data):
        return ModelName.objects.create(**validated_data)

    def update(self, instance, validated_data):
        instance.field_name = validated_data.get('field_name', instance.field_name)
        instance.save()
        return instance
```

*Plain Serializer — manual fields + manual create/update logic*

### Sample Code

```
# serializers.py
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'email', 'age', 'is_active', 'created_at']
        read_only_fields = ['created_at']
```

*ModelSerializer for the Student model*

#### Why we use Serializers

Python/Django objects (querysets, model instances) cannot be sent directly over HTTP as JSON. Serializers bridge that gap in both directions: model → JSON (for responses) and JSON → validated Python data → model (for incoming requests). They also handle field-level and object-level **validation**, so invalid data never reaches the database.

### 3 View vs ViewSet

**Definition — View:** A View (e.g. DRF's `APIView` or the generic views like `ListAPIView`) is a Python class/function that handles a single HTTP method per URL — you write `get()`, `post()`, `put()`, `delete()` etc. explicitly, and each one is normally wired to its own URL.

**Definition — ViewSet:** A ViewSet groups the logic for a related set of views (list, retrieve, create, update, destroy) for one model into a **single class**. It does not directly bind to URLs itself — it is registered with a **Router**, which auto-generates all the standard URL patterns for it.

#### Basic Syntax

```
# APIView style
from rest_framework.views import APIView
from rest_framework.response import Response

class NameAPIView(APIView):
    def get(self, request):
        ...
        return Response(data)

    def post(self, request):
        ...
        return Response(data)
```

*APIView — one method per HTTP verb*

```
# ViewSet style
from rest_framework import viewsets

class NameViewSet(viewsets.ModelViewSet):
    queryset = ModelName.objects.all()
    serializer_class = NameSerializer
```

*ModelViewSet — list/retrieve/create/update/destroy all built-in*

#### Sample Code

```
# views.py – APIView approach
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .models import Student
from .serializers import StudentSerializer

class StudentListAPIView(APIView):
    def get(self, request):
        students = Student.objects.all()
        serializer = StudentSerializer(students, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = StudentSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

*Equivalent logic written as an APIView (more code, more control)*

```
# views.py – ViewSet approach
from rest_framework import viewsets
from .models import Student
from .serializers import StudentSerializer

class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer
    # list, retrieve, create, update, partial_update, destroy
    # are ALL provided automatically
```

*Same CRUD functionality as above, written in 4 lines with a ViewSet*

## View vs ViewSet — Quick Comparison

| Aspect       | View (APIView)                  | ViewSet                                      |
|--------------|---------------------------------|----------------------------------------------|
| URL mapping  | Manual — one path() per view    | Automatic — via Router                       |
| CRUD methods | You write get/post/put/delete   | list/create/retrieve/update/destroy built-in |
| Code amount  | More boilerplate                | Compact, less repetition                     |
| Control      | Fine-grained, explicit          | Convention-based, still overridable          |
| Best for     | Custom / non-standard endpoints | Standard CRUD REST APIs                      |

### Why we use Views / ViewSets

This is the layer that actually receives the HTTP request, decides what to do with it (query the DB, validate input, call the serializer) and returns an HTTP response. Plain Views give full manual control for one-off/custom endpoints. ViewSets remove repeated CRUD boilerplate when a model needs the standard 5 operations, and they pair with Routers to auto-generate URLs — saving time on typical REST resources.

## 4 Router

**Definition:** A Router automatically generates the standard set of URL patterns (list, detail, create, update, delete) for a ViewSet, so you don't have to write `path()` entries manually for each action. DRF provides `SimpleRouter` and `DefaultRouter` (the latter also adds a browsable API root view).

### Basic Syntax

```
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('prefix', ViewSetName, basename='name')

urlpatterns = router.urls
```

*Registering a ViewSet with a router*

### Sample Code

```
# urls.py
from rest_framework.routers import DefaultRouter
from .views import StudentViewSet

router = DefaultRouter()
router.register('students', StudentViewSet, basename='student')

urlpatterns = router.urls

# This automatically creates:
# GET/POST      /students/
# GET/PUT/PATCH/DELETE /students/<id>/
```

*DefaultRouter auto-generating all Student CRUD URLs*

#### Why we use Routers

Without a router, you would need to manually write a separate URL path for every action (list, retrieve, create, update, delete) of every ViewSet. A router inspects the ViewSet and wires all of these up in one line, keeping `urls.py` short, consistent, and less error-prone — especially valuable as the number of models/endpoints grows.

## 5 URL

**Definition:** The `URLconf` (usually `urls.py`) maps a URL path to a view. Django checks incoming request paths against this list, top to bottom, and calls the matching view. For plain `APIViews` you map URLs manually with `path()`; for `ViewSet`s you normally let a Router generate them.

### Basic Syntax

```
from django.urls import path

urlpatterns = [
    path('route/', ViewName.as_view(), name='route-name'),
]
```

*Manual URL mapping for a class-based View*

### Sample Code

```
# urls.py – manual (for APIView)
from django.urls import path
from .views import StudentListAPIView

urlpatterns = [
    path('students/', StudentListAPIView.as_view(), name='student-list'),
]
```

*Manual path for an APIView*

```
# project-level urls.py – including app + router urls
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('students.urls')),  # app urls (router.urls)
]
```

*Project urls.py including the app's router-generated urls*

#### Why we use URLs

URLs are the entry point of every request — they decide which piece of view/viewset code runs for a given path and HTTP method. Clean, well-named URL routing makes the API predictable and RESTful (e.g. `/api/students/`, `/api/students/1/`).

## 6 Workflow — How They Connect

A single API request flows through all five pieces in order. Below is the request (incoming) and response (outgoing) cycle for, e.g., GET /api/students/.

|                            |                                                                                                                                                                        |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Client Request</b>   | A client (browser, Postman, frontend app) sends an HTTP request, e.g. GET /api/students/.                                                                              |
| <b>2. URL / Router</b>     | Django's URLconf matches the path. If a Router is used, it identifies the correct ViewSet action (list, retrieve, etc.) for that path + method.                        |
| <b>3. View / ViewSet</b>   | The matched View or ViewSet method runs. It fetches data from the database using the Model (e.g. Student.objects.all()).                                               |
| <b>4. Serializer</b>       | The queryset/model instance(s) are passed into the Serializer, which converts them into a JSON-ready Python dict/list (and validates data on the way in for POST/PUT). |
| <b>5. Model / Database</b> | The Model defines the table structure the ORM queries against — it's used by the view during step 3 to talk to the database.                                           |
| <b>6. Response</b>         | The View/ViewSet wraps the serialized data in an HTTP Response, which DRF renders as JSON and sends back to the client.                                                |

### Diagram (text form)

```
Client
|
| HTTP Request (GET/POST/PUT/DELETE)
v
URL / Router -----> matches path to a View/ViewSet action
|
v
View / ViewSet -----> talks to Model (ORM) to fetch/save data
|                               |
|                               v
|                               Database
v
Serializer -----> converts Model data <-> JSON, validates input
|
v
Response (JSON)
|
v
Client (receives data)
```

*Full request/response cycle across all 5 pieces*

**In short:** URL/Router decides *which* code runs → View/ViewSet decides *what* to do (using the Model to hit the DB) → Serializer decides *how* the data looks going in/out → Response is sent back to the client.